

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What will happen?

```
def fun(n):
    if n == 0:
        return 1
    return n * fun(n)
```

```
print(fun(3))
```

- A)3
- B)RecursionError
- C)6
- D)TypeError

Q2. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

- A)24
- B)0
- C)9
- D)10

Q3. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        return x + 5
```

```
    return inner
```

```
f = outer()
print(f())
```

- A)None
- B)15

C)10

D)NameError

Q4. What will happen?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal y
        y = 20
```

```
    inner()
```

```
outer()
```

- A)10
- B)NameError
- C)20
- D)SyntaxError

Q5. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

- A)ZeroDivisionError is not an exception
- B)Exception is a superclass of ZeroDivisionError
- C)ZeroDivisionError occurs before Exception
- D)except blocks execute randomly

Q6. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A)TypeError
- B)20
- C)SyntaxError



D)30

Q7. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

- A)10
- B)20
- C)RuntimeError
- D)None

Q8. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

- A)cleanup
- 100
- B)negative
cleanup
- C)cleanup
- 5
- D)100
cleanup

Q9. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:
```

print("D")

print(fun())

- A)A
- B
- D
- 10
- B)A
- B
- 10
- D
- C)A
- B
- C
- D
- 10
- D)A
- C
- D
- 20

**Q10. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)A
- B)A followed by B
- C)B
- D)Nothing

Q11. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:
```



```
print(type(e).__name__)
print(e)
```

A) Nothing, because custom exceptions cannot be caught

B) InvalidAgeError

15

C) Exception

Age is too low

D) InvalidAgeError

Age is too low

Q12. What will be printed?

```
try:
```

```
    print(10 / 0)
```

```
except ZeroDivisionError:
```

```
    print("A")
```

```
except Exception:
```

```
    print("B")
```

A) A B

B) Nothing

C) B

D) A

Q13. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

A) Wrong Value

B) Nothing

C) ValueError

D) e

Q14. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

A) 15 5

B) 5 5

C) 15 15

D) 5 15

Q15. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

```
print(fun(3))
```

A) 3

B) 5

C) None

D) 6

Q16. What happens when this code executes?

```
def fun():
```

```
    print(variable)
```

```
    variable = 50
```

```
variable = 10
```

```
fun()
```

A) UnboundLocalError

B) 10

C) NameError

D) 50

Q17. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A) A C

B) B C

C) B

D) A

Q18. What will happen?

```
def fun(a, b=10):
```

```
    return a + b
```

```
print(fun(b=5, a=20))
```

A) 15

B) TypeError

C) SyntaxError

D) 25



Q19. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A)5
- B)5.0
- C)25
- D)sqrt(25)

Q20. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Hello None
- B)Hello
- C>Error
- D)None

Q21. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)
```

```
print(b)
```

- A)[10, 10]
- [10]
- B)[10]
- [10]
- C)[10]
- [10, 10]
- D>Error

Q22. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

- A)0
- B)TypeError

C)None

D)NameError

Q23. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)SyntaxError

B)A

C)B

D)A B

Q24. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)No Output

B)A, followed by propagation of the same exception

C)B

D)A, and execution ends normally

Q25. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():
```

```
    raise MyError("Something went wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)Exception

B)MyError

C)TypeError

D)Something went wrong

Q26. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```



- A)15
- B)Error
- C)5
- D)10

Q27. What will be printed?

```
x = 10
```

```
def outer():  
    x = 20  
  
    def inner():  
        print(x)  
  
    x = 30  
    inner()
```

outer()

- A)10
- B)NameError
- C)30
- D)20

Q28. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

- A)10
- Hello
- B)x
- display
- C)Error
- D)Only Hello

Q29. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)Nothing
- B)A and B
- C)A
- D)B

Q30. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.imports
- B).__module__
- C)sys.path
- D)sys.modules

Q31. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

fun()

- A)B
- B)A
- C)Nothing
- D)A B

Q32. What will normally happen?

```
import math  
import math
```

- A)ImportError
- B)The module is normally loaded and executed twice
- C)The module is normally loaded once and reused
- D)SyntaxError

Q33. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6.0
- B)6
- C)36
- D)NameError



Q34. What is the purpose of this statement?

```
if __name__ == "__main__":
```

- A)It prevents syntax errors
- B)It prevents all imports
- C)It identifies code intended to run when the file is executed directly
- D)It creates the main function

Q35. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

- A)Handled
- B)NameError
- C)TypeError
- D)RuntimeError cannot propagate through functions

Q36. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
- Module loaded

10

B)Module loaded

10

C)ImportError

D)10

10

Q37. What happens when this function is called?

```
def fun(a=10, b):  
    return a + b
```

- A)NameError
- B)TypeError
- C)It executes correctly
- D)SyntaxError

Q38. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

- A)5
- B)15
- C)10
- D)UnboundLocalError

Q39. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)Python will always reject it
- B)The local file can shadow the intended standard-library module
- C)random cannot be imported
- D)Python automatically renames the local file

Q40. What is the output?

```
def fun(x):
```



```
return x * 2
```

```
a = fun  
print(a(5))
```

- A)10
- B)Error
- C)func(5)
- D)5

Q41. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)B C
- B)A C
- C)A
- D)B

Q42. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

- A)UnboundLocalError
- B)10
- C)20
- D)NameError

Q43. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)First provides the module namespace; second imports sqrt directly
- B)There is no difference
- C)First imports only sqrt
- D)Both import only sqrt

Q44. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)10
- B)20
- C)30
- D)-10

Q45. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[2, 4]
- B)[4, 8]
- C)[1, 3, 5]
- D)[2, 4, 6, 8, 10]

Q46. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

- A)30
- 30
- B)20
- 10



C)20

30

D)30

20

Q47. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)None

B)10

C)NameError

D)20

Q48. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

1

3

B)1

2

3

4

C)0

1

0

2

D)1

1

1

1

Q49. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)10 only

B)10 followed by Finally

C)Finally followed by 10

D)Finally only

Q50. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)20

C)SyntaxError

D)TypeError

